

Laporan Tugas Besar

IF2211 Strategi Algoritma

Implementasi Strategi Greedy pada Bot Battlecode 2025

Semester II Tahun 2025/2026



Kelompok DendengBalado

Gabriella Botimada Lubis 13524006

Agatha Tatianingseto 13524008

Junior Natra Situmorang 13524055

Program Studi Teknik Informatika

Sekolah Teknik Elektro dan Informatika

Institut Teknologi Bandung

Daftar Isi

1	Deskripsi Tugas	3
1.1	Latar Belakang	3
1.2	Komponen Permainan	3
1.2.1	Objectives	3
1.2.2	Aturan Tiebreaker	3
1.2.3	Peta	3
1.2.4	Jarak Pandang dan Pergerakan	4
1.2.5	Tower dan Ruins	4
1.2.6	Pola Sumber Daya Khusus	4
1.2.7	Cat	4
1.2.8	Chips	4
1.2.9	Robot	4
1.2.10	Jenis Robot	4
1.2.11	Towers	5
1.2.12	Serangan	5
1.2.13	Penambahan	5
1.2.14	Komunikasi	5
1.2.15	Penanda	5
1.2.16	Action dan Cooldown	5
2	Landasan Teori	7
2.1	Algoritma Greedy	7
2.2	Cara Kerja Program	7
3	Aplikasi Strategi Greedy	8
3.1	Mapping Persoalan	8
3.2	Mapping Permasalahan Battlecode ke Komponen Algoritma Greedy	8
3.2.1	Himpunan Kandidat	8
3.2.2	Himpunan Solusi	10
3.2.3	Fungsi Seleksi	10
3.2.4	Fungsi Kelayakan	11
3.2.5	Fungsi Solusi	11
3.2.6	Fungsi Objektif	12
3.3	Alternatif Solusi Greedy 1	12
3.3.1	Heuristik yang Digunakan	12
3.3.2	Fokus Utama Setiap Unit	12
3.3.3	Logika Greedy Sederhana	13
3.3.4	Strategi Jalan	13
3.3.5	Strategi Target Cat	14
3.3.6	Manajemen Sumber Daya	16
3.3.7	Spawn Strategy	16
3.3.8	Pemanfaatan Pola Khusus (SRP dan Tower)	16

3.4	Alternatif Solusi Greedy 2	17
3.5	Alternatif Solusi Greedy 3	18
3.5.1	State Robot	19
3.5.2	Alur Setiap Robot	20
3.5.3	Alur Robot Setiap Phase	21
3.5.4	Threshold	21
3.6	Analisis Efisiensi	22
3.6.1	Alternatif Solusi Greedy 1	22
3.6.2	Alternatif Solusi Greedy 2	23
3.6.3	Alternatif Solusi Greedy 3	24
3.7	Strategi yang Dipilih	24
4	Implementasi dan Pengujian	26
4.1	Pseudocode	26
4.2	Struktur Data dan Method	26
4.2.1	G.java	26
4.2.2	RobotPlayer.java	27
4.2.3	Soldier.java	27
4.2.4	Splasher.java	30
4.2.5	Mopper.java	31
4.2.6	Tower.java	32
4.2.7	Nav.java	33
4.2.8	MapMemory.java	34
4.2.9	Rand.java	36
4.3	Pengujian	36
4.4	Analisis Hasil	36
5	Kesimpulan dan Saran	37

1 Deskripsi Tugas

1.1 Latar Belakang

Battlecode adalah permainan strategi waktu nyata di mana Anda akan menulis kode untuk pemain otonom. Pemain Anda harus mengelola pasukan robot secara strategis dan mengendalikan cara robot-robot Anda bekerja sama untuk mengalahkan tim lawan. Sebagai peserta, Anda akan belajar menggunakan kecerdasan buatan, penelusuran jalur, algoritma terdistribusi, dan komunikasi untuk membuat pemain Anda sesaing mungkin. Namun, Anda hanya memiliki sumber daya komputasi terbatas per giliran, jadi Anda harus merencanakan strategi, mengoptimalkan, dan berkompromi sesuai kebutuhan. Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan strategi greedy dalam membuat bot ini.

1.2 Komponen Permainan

Komponen-komponen dari permainan ini antara lain:

1.2.1 Objectives

Objective utama adalah mewarnai peta. Tim pertama yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai di peta akan memenangkan permainan. Tim juga bisa menang dengan cara menghancurkan semua robot dan menara milik tim lawan.

1.2.2 Aturan Tiebreaker

Jika setelah 2000 ronde tidak ada tim yang berhasil mewarnai lebih dari 70% petak yang dapat diwarnai, permainan akan langsung berakhir. Penentuan pemenang dilakukan dengan urutan tiebreaker berikut:

- Luas area yang sudah diwarnai di peta
- Jumlah menara sekutu yang masih hidup
- Total jumlah uang (chips)
- Jumlah total cat (paint) yang dimiliki semua robot dan menara
- Jumlah robot sekutu yang masih hidup
- Jika masih seri, tim dipilih secara acak (uniform random)

1.2.3 Peta

Setiap permainan dimainkan di sebuah peta berbentuk grid persegi panjang 2 dimensi, dengan ukuran antara 20×20 hingga 60×60 petak (inklusif). Titik kiri bawah peta memiliki koordinat (0,0); koordinat bertambah ke arah Timur (kanan) dan Utara (atas). Lokasi di peta direpresentasikan dengan objek `MapLocation` yang menyimpan nilai x dan y. Untuk menjaga keseimbangan permainan, peta dijamin simetris melalui rotasi atau refleksi.

1.2.4 Jarak Pandang dan Pergerakan

Robot dapat melihat fitur peta dan robot lain dalam jarak hingga $\sqrt{20}$ atau sekitar 4.47 petak. Unit tidak dapat bergerak melewati menara atau robot lain. Peta juga dapat memiliki tembok (walls) yang tidak dapat dilewati maupun diwarnai. Jumlah tembok maksimal adalah 20% dari seluruh peta. Selain tembok, terdapat reruntuhan (ruins) yang juga tidak dapat dilewati atau diwarnai oleh robot.

1.2.5 Tower dan Ruins

Menara berfungsi sebagai zona spawn tim sekaligus sebagai unit yang dapat menyerang musuh. Untuk membangun menara, tim harus mewarnai pola tertentu pada reruntuhan yang tersedia di peta. Setiap permainan dimulai dengan sejumlah reruntuhan yang sudah tersedia di peta. Reruntuhan tidak dapat dipindahkan atau dihancurkan. Pusat reruntuhan berjarak minimal lima petak satu sama lain, dan area 5×5 di sekitarnya dijamin bebas dari tembok. Setiap tim memulai permainan dengan dua menara, yaitu satu menara penghasil cat dan satu menara penghasil uang.

1.2.6 Pola Sumber Daya Khusus

Pola sumber daya khusus atau Special Resource Pattern (SRP) merupakan pola yang dapat meningkatkan produksi sumber daya tim. Jika sebuah pola berhasil diselesaikan dan tetap utuh selama 50 ronde, pola tersebut akan menjadi aktif. Setiap pola aktif akan memberikan tambahan tiga sumber daya per giliran pada semua menara tambang sekutu.

1.2.7 Cat

Cat merupakan sumber daya utama dalam permainan. Cat digunakan untuk mengklaim wilayah pada peta dan menentukan skor tim. Setiap robot dan menara memiliki kapasitas penyimpanan cat yang berbeda-beda. Jika jumlah cat yang dimiliki robot terlalu sedikit, robot akan mengalami peningkatan cooldown atau bahkan tidak dapat melakukan aksi.

1.2.8 Chips

Chips digunakan sebagai sumber daya untuk membangun unit dan menara. Chips dihasilkan setiap ronde oleh menara penghasil uang. Setiap tim memulai permainan dengan 2500 chips yang dapat digunakan oleh semua robot sekutu.

1.2.9 Robot

Semua robot dapat melakukan berbagai aksi seperti bergerak, merasakan lingkungan sekitar, dan berkomunikasi dengan robot lain. Permainan berjalan secara bergiliran dalam ronde, dan setiap robot akan menjalankan kode yang ditulis oleh pemain. Setiap robot memiliki batas penggunaan bytecode pada setiap giliran. Jika bytecode habis sebelum robot selesai menjalankan programnya, maka giliran robot tersebut akan langsung berakhir.

1.2.10 Jenis Robot

Terdapat tiga jenis robot utama dalam permainan: Soldier, Splasher, dan Mopper. Masing-masing robot memiliki kemampuan, kapasitas cat, serta biaya produksi yang berbeda.

1.2.11 Towers

Semua menara dibangun dengan prosedur yang sama, yaitu dengan mewarnai pola tertentu pada area 5×5 di sekitar reruntuhan. Langkah pembangunan menara adalah:

1. Memanggil fungsi `markTowerPattern()`
2. Mewarnai pola yang ditentukan
3. Memanggil `completeTowerPattern()` untuk memunculkan menara

1.2.12 Serangan

Menara dapat melakukan dua jenis serangan:

- Serangan satu petak yang menargetkan satu unit musuh
- Serangan area efek (Area of Effect)

Dalam satu giliran, menara dapat melakukan satu serangan satu petak dan satu serangan area.

1.2.13 Penambangan

Setiap giliran, menara penghasil uang dan menara penghasil cat akan menambang sumber daya secara pasif. Cat yang dihasilkan disimpan di dalam menara dan dapat diambil oleh robot sekutu, sedangkan chips dapat langsung digunakan oleh semua unit sekutu.

1.2.14 Komunikasi

Robot tidak dapat berbagi variabel secara langsung sehingga koordinasi antar robot harus dilakukan melalui komunikasi. Komunikasi dapat dilakukan melalui dua cara:

- Pesan (Messages)
- Penanda (Markers)

Pesan dapat berisi informasi apa pun tetapi dibatasi hingga 4 byte. Robot dapat mengirim maksimal satu pesan per ronde, sedangkan menara dapat mengirim hingga dua puluh pesan per ronde.

1.2.15 Penanda

Penanda merupakan informasi yang dapat ditempatkan pada peta oleh robot untuk memberikan petunjuk kepada robot sekutu lainnya mengenai pola atau area tertentu. Penanda hanya dapat dilihat oleh robot sekutu dan tidak memengaruhi kepemilikan wilayah pada peta.

1.2.16 Action dan Cooldown

Robot dapat melakukan berbagai aksi untuk berinteraksi dengan lingkungan permainan. Beberapa aksi memiliki cooldown sehingga tidak dapat dilakukan secara berulang dalam waktu singkat. Aksi utama yang dapat dilakukan robot meliputi:

- Menyerang atau mewarnai petak
- Bergerak ke petak lain

- Menandai petak
- Mentransfer cat
- Menarik cat dari menara
- Spawn robot dari menara
- Mengirim pesan
- Menghancurkan diri sendiri

2 Landasan Teori

2.1 Algoritma Greedy

Algoritma greedy adalah metode penyelesaian masalah yang bekerja secara bertahap (step by step). Pada setiap langkah, algoritma ini memilih keputusan yang dianggap paling optimal pada saat itu tanpa mempertimbangkan konsekuensi dari keputusan tersebut terhadap langkah-langkah berikutnya.

—————GAMBAR—————

Prinsip utama dari algoritma greedy adalah mengambil pilihan terbaik yang tersedia pada kondisi saat ini. Pendekatan ini sering disebut sebagai prinsip “rakus” (greedy), yaitu memilih keuntungan terbesar yang dapat diperoleh segera, dengan asumsi bahwa pilihan lokal terbaik tersebut pada akhirnya akan menghasilkan solusi global yang optimal.

Secara umum, algoritma greedy memiliki dua karakteristik utama, yaitu:

a. **Memilih solusi optimal lokal pada setiap langkah**

Pada setiap tahap penyelesaian, algoritma akan memilih pilihan terbaik yang tersedia pada saat itu tanpa mempertimbangkan kemungkinan dampaknya pada langkah selanjutnya.

b. **Mengharapkan tercapainya solusi optimal global**

Dengan memilih solusi terbaik pada setiap langkah, algoritma berharap bahwa rangkaian pilihan lokal tersebut akan menghasilkan solusi optimal secara keseluruhan.

Pendekatan greedy banyak digunakan dalam berbagai permasalahan optimasi karena memiliki implementasi yang sederhana serta efisiensi waktu yang baik. Namun, metode ini tidak selalu menghasilkan solusi optimal untuk semua jenis permasalahan, terutama pada kasus yang tidak memenuhi sifat greedy choice property dan optimal substructure. Algoritma greedy memiliki beberapa sifat utama yang mendukung penggunaannya dalam menyelesaikan berbagai permasalahan optimasi. Dua sifat penting dari algoritma greedy adalah sebagai berikut.

—————GAMBAR—————

2.2 Karakteristik Algoritma Greedy

a. **Greedy Choice Property**

Greedy Choice Property merupakan sifat yang menyatakan bahwa solusi optimal dari suatu permasalahan dapat diperoleh dengan memilih keputusan terbaik secara lokal pada setiap langkah. Dengan kata lain, pada setiap tahap penyelesaian masalah, algoritma memilih opsi yang memberikan keuntungan paling besar pada saat itu tanpa mempertimbangkan kemungkinan keputusan di masa depan.

Jika suatu permasalahan memiliki sifat ini, maka pendekatan greedy dapat digunakan karena pilihan lokal terbaik pada setiap langkah akan tetap mengarah pada solusi optimal secara keseluruhan.

b. **Optimal Substructure**

Optimal Substructure adalah sifat yang menyatakan bahwa solusi optimal dari suatu permasalahan dapat dibentuk dari solusi optimal dari subpermasalahan yang lebih kecil. Artinya, jika suatu masalah dipecah menjadi beberapa bagian yang lebih kecil, maka penyelesaian terbaik untuk keseluruhan

masalah dapat diperoleh dengan menggabungkan solusi terbaik dari masing-masing subpermasalahan tersebut.

Sifat ini sangat penting dalam algoritma greedy karena memungkinkan proses penyelesaian dilakukan secara bertahap. Dengan menyelesaikan sub-permasalahan secara optimal pada setiap langkah, solusi akhir yang diperoleh juga akan menjadi solusi optimal untuk permasalahan secara keseluruhan.

2.3 Prosedur Perancangan Algoritma Greedy

Dalam merancang algoritma greedy untuk menyelesaikan suatu persoalan optimasi, terdapat beberapa langkah atau prosedur yang perlu dilakukan agar pendekatan greedy dapat diterapkan secara tepat. Prosedur tersebut meliputi hal-hal berikut.

- **Merumuskan persoalan optimasi dengan pilihan greedy**

Langkah pertama adalah merumuskan persoalan optimasi sedemikian rupa sehingga pada setiap tahap penyelesaian terdapat pilihan yang dapat diambil secara greedy. Persoalan dipecah menjadi serangkaian keputusan yang dilakukan secara bertahap, di mana setiap keputusan memilih opsi yang dianggap paling menguntungkan pada saat itu. Setelah suatu pilihan diambil, persoalan yang tersisa akan menjadi sub-persoalan yang lebih kecil untuk diselesaikan pada langkah berikutnya.

- **Menunjukkan adanya optimal substructure**

Langkah selanjutnya adalah menunjukkan bahwa persoalan tersebut memiliki sifat optimal substructure. Artinya, solusi optimal dari persoalan utama dapat diperoleh dengan menggabungkan solusi optimal dari sub-persoalan yang dihasilkan pada setiap langkah. Dengan adanya sifat ini, penyelesaian secara bertahap menggunakan pendekatan greedy tetap dapat menghasilkan solusi yang optimal.

- **Membuktikan adanya solusi greedy-optimal**

Langkah terakhir adalah membuktikan secara matematis bahwa selalu terdapat solusi optimal yang dapat diperoleh dengan mengikuti pilihan greedy pada setiap langkah. Pembuktian ini penting untuk memastikan bahwa keputusan lokal yang diambil oleh algoritma greedy tidak akan menghalangi tercapainya solusi optimal secara keseluruhan.

2.4 Cara Kerja Program

3 Aplikasi Strategi Greedy

3.1 Mapping Persoalan

3.2 Mapping Permasalahan Battlecode ke Komponen Algoritma Greedy

Permainan Battlecode 2025 mengharuskan setiap robot dan menara untuk mengambil keputusan pada setiap ronde permainan. Keputusan tersebut meliputi berbagai aksi seperti bergerak, menyerang, mewarnai wilayah, membersihkan cat musuh, maupun melakukan spawn robot baru. Karena setiap unit memiliki keterbatasan bytecode dan waktu komputasi, maka pemilihan aksi harus dilakukan secara efisien. Pendekatan algoritma greedy digunakan untuk memilih aksi yang dianggap paling menguntungkan berdasarkan kondisi permainan saat itu. Namun, setiap jenis unit pada Battlecode memiliki fungsi dan kemampuan yang berbeda. Oleh karena itu, proses pengambilan keputusan greedy tidak dapat disamaratakan untuk semua unit. Setiap jenis robot dan tower memiliki himpunan kandidat dan prioritas aksi yang berbeda. Untuk memformalkan permasalahan ini ke dalam kerangka algoritma greedy, diperlukan definisi komponen-komponen berikut: himpunan kandidat, himpunan solusi, fungsi seleksi, fungsi kelayakan, fungsi solusi, dan fungsi objektif.

3.2.1 Himpunan Kandidat

Himpunan kandidat merupakan kumpulan seluruh aksi yang dapat dilakukan oleh suatu unit pada suatu ronde permainan. Karena setiap jenis robot memiliki kemampuan yang berbeda, maka himpunan kandidat untuk setiap robot juga berbeda.

Jenis Robot	Himpunan Kandidat (Aksi yang Dapat Dilakukan)
Soldier	• Bergerak ke salah satu dari delapan arah • Menyerang robot musuh dalam radius serangan • Mewarnai petak netral • Menghapus cat musuh • Bergerak menuju wilayah yang belum dicat • Membantu menyelesaikan pola pembangunan tower • Membantu menyelesaikan pola resource
Splasher	• Bergerak ke salah satu arah • Melakukan serangan area (splash attack) • Mewarnai beberapa petak sekaligus melalui serangan area • Bergerak menuju kelompok musuh • Bergerak menuju wilayah yang banyak dikuasai musuh
Mopper	• Bergerak menuju wilayah yang dicat musuh • Menghapus cat musuh • Melakukan serangan mop swing • Mentransfer cat ke robot sekutu • Mengambil cat dari menara • Bergerak menuju robot sekutu yang kekurangan cat

Table 1: Himpunan kandidat untuk setiap jenis robot

Selain robot, tower juga memiliki himpunan kandidat yang berbeda tergantung pada jenis tower yang dimiliki oleh tim.

Jenis Tower	Himpunan Kandidat
Money Tower	• Spawn robot baru • Menyerang musuh dalam jangkauan • Melakukan serangan area • Upgrade tower
Paint Tower	• Spawn robot • Menyerang robot musuh • Menyediakan cat bagi robot sekutu • Upgrade tower
Defense Tower	• Menyerang robot musuh • Melakukan serangan area • Melindungi wilayah sekitar • Upgrade tower

Table 2: Himpunan kandidat untuk setiap jenis tower

3.2.2 Himpunan Solusi

Himpunan solusi merupakan kumpulan aksi yang dipilih oleh bot untuk dilakukan oleh seluruh unit pada suatu ronde permainan. Setiap robot dan tower akan memilih satu aksi dari himpunan kandidat yang tersedia berdasarkan prioritas strategi yang digunakan. Contoh himpunan solusi pada suatu ronde dapat berupa:

- Soldier bergerak menuju wilayah netral
- Soldier lainnya menyerang robot musuh
- Splasher melakukan serangan area
- Mopper membersihkan cat musuh
- Tower melakukan spawn robot baru

Gabungan dari seluruh aksi tersebut membentuk keputusan strategi bot pada ronde tersebut.

3.2.3 Fungsi Seleksi

Fungsi seleksi digunakan untuk menentukan aksi mana yang memiliki prioritas tertinggi dari himpunan kandidat. Prioritas aksi berbeda untuk setiap jenis robot karena setiap robot memiliki peran yang berbeda dalam permainan.

Jenis Robot	Prioritas Aksi (Greedy Selection)
Soldier	1. Menyerang musuh dalam jangkauan 2. Mewarnai petak netral 3. Menghapus cat musuh 4. Bergerak menuju wilayah yang belum dicat 5. Membantu menyelesaikan pola tower
Splasher	1. Menyerang area dengan banyak musuh 2. Menyerang wilayah dengan banyak cat musuh 3. Bergerak menuju area konflik 4. Bergerak menuju wilayah musuh
Mopper	1. Menghapus cat musuh 2. Mentransfer cat ke robot sekutu 3. Mengambil cat dari tower 4. Bergerak menuju wilayah dengan banyak cat musuh

Table 3: Fungsi seleksi untuk setiap jenis robot

3.2.4 Fungsi Kelayakan

Fungsi kelayakan digunakan untuk memeriksa apakah suatu aksi dapat dilakukan tanpa melanggar aturan permainan. Beberapa kondisi yang diperiksa antara lain:

- Robot hanya dapat bergerak jika petak tujuan tidak ditempati robot lain dan bukan tembok.
- Robot hanya dapat menyerang jika musuh berada dalam radius serangan.
- Robot hanya dapat mewarnai petak jika memiliki cat yang cukup.
- Robot hanya dapat mentransfer cat jika robot sekutu berada dalam jarak transfer.
- Tower hanya dapat melakukan spawn jika memiliki sumber daya yang cukup dan petak spawn kosong.

Jika suatu kandidat tidak memenuhi kondisi tersebut, maka kandidat tersebut tidak dapat dipilih sebagai bagian dari solusi.

3.2.5 Fungsi Solusi

Fungsi solusi menentukan kapan proses pemilihan aksi dianggap selesai. Dalam konteks Battlecode, solusi dianggap selesai ketika setiap unit telah menentukan aksi yang akan dilakukan pada ronde tersebut. Dengan demikian, proses pemilihan aksi berhenti ketika:

- setiap robot telah memilih satu aksi dari himpunan kandidat
- setiap tower telah menentukan aksinya
- tidak ada aksi tambahan yang dapat dilakukan pada ronde tersebut

3.2.6 Fungsi Objektif

Fungsi objektif mendefinisikan tujuan utama dari strategi greedy yang digunakan oleh bot. Berdasarkan aturan permainan Battlecode, tujuan utama permainan adalah mewarnai lebih dari 70% wilayah peta atau menghancurkan seluruh unit musuh. Dengan demikian, fungsi objektif dari strategi greedy dapat dirumuskan sebagai berikut:

- Memaksimalkan jumlah petak wilayah yang berhasil dicat oleh tim.
- Meminimalkan wilayah yang dikuasai oleh tim lawan.
- Memaksimalkan jumlah menara sekutu yang bertahan.
- Memaksimalkan jumlah robot sekutu yang aktif.
- Meminimalkan kerugian unit akibat serangan musuh.

Strategi greedy yang digunakan oleh bot akan berusaha memilih aksi yang memberikan keuntungan terbesar terhadap tujuan-tujuan tersebut pada setiap ronde permainan.

3.3 Alternatif Solusi Greedy 1

3.3.1 Heuristik yang Digunakan

Strategi greedy pertama yang dirancang berfokus pada dominasi wilayah permainan dengan tetap menjaga keberlangsungan ekonomi tim. Tujuan utama dari strategi ini adalah menguasai wilayah seluas mungkin sambil memastikan suplai *paint* dan *chips* tidak terputus. Dominasi wilayah tanpa dukungan ekonomi yang stabil akan menyebabkan robot tidak dapat bergerak karena kehabisan *paint* atau tim tidak dapat melakukan spawn robot baru ketika unit sebelumnya hancur. Oleh karena itu, strategi ini menggunakan dua pendekatan heuristik utama:

- **Greedy on Coverage** — memaksimalkan jumlah petak wilayah yang berhasil dicat.
- **Greedy on Resource** — memanfaatkan *ruins* dan *SRP* untuk meningkatkan ekonomi tim.

Fungsi seleksi dasar yang digunakan untuk memilih aksi adalah:

$$Score = (10 \times JumlahPetakWarnaBaru) + (50 \text{ jika mendekati Ruins})$$

Robot akan memilih aksi dengan skor tertinggi pada kondisi permainan saat itu.

3.3.2 Fokus Utama Setiap Unit

Setiap jenis robot dan tower memiliki peran yang berbeda dalam mendukung strategi ini.

Unit	Fokus Utama
Splasher	Menjadi unit utama untuk melakukan ekspansi wilayah secara besar-besaran pada area kosong atau wilayah musuh.
Soldier	Digunakan untuk menyelesaikan pola pembangunan tower secara presisi pada lokasi ruins.
Mopper	Menjaga area sekitar tower agar tetap bersih dari cat musuh serta mempertahankan wilayah yang sudah dikuasai.
Tower	Pada awal permainan berfokus pada pembangunan Money Tower untuk menghasilkan chips, kemudian Paint Tower untuk mendukung ekspansi Splasher.

3.3.3 Logika Greedy Sederhana

Logika greedy yang digunakan pada strategi ini dapat diringkas sebagai berikut:

- Jika terdapat *ruins* atau SRP di dekat robot, maka robot memprioritaskan bergerak menuju lokasi tersebut.
- Jika tidak terdapat target sumber daya, robot akan mencari petak netral atau wilayah musuh terdekat untuk diwarnai.
- Robot selalu memilih aksi yang memberikan nilai tertinggi terhadap kontrol wilayah atau peningkatan sumber daya.

3.3.4 Strategi Jalan

Pergerakan robot ditentukan menggunakan fungsi skor berikut:

$$ScoreMove(d) = \alpha \cdot CoverageGain(d) + \beta \cdot ResourceGain(d) + \gamma \cdot Safety(d) + \delta \cdot Connectivity(d) - \epsilon \cdot CostRisk(d)$$

Robot akan memilih arah pergerakan d dengan skor tertinggi.

Keterangan variabel

- CoverageGain : peluang robot dapat mengecat lebih banyak petak netral atau musuh.
- ResourceGain : bonus jika pergerakan mendekati ruin, SRP, atau paint tower sekutu.
- Safety : bonus jika robot berada pada wilayah sekutu atau menjauhi tower musuh.
- Connectivity : bonus jika pergerakan menjaga konektivitas jalur cat sekutu.
- CostRisk : penalti jika pergerakan masuk wilayah musuh dengan paint rendah, mendekati tower musuh, atau menuju area buntu.

Robot menggunakan beberapa mode operasi: EXPAND, BUILD_TOWER, BUILD_SRP, DEFEND_PAINT_LINE, REFILL, RETREAT

Table 4: Strategi jalan untuk masing-masing robot

Robot	Strategi Jalan
Splasher	• Mengenumerasi semua petak yang mungkin dijadikan pusat serangan dalam jangkauan aksi. • Menghitung jumlah petak netral atau petak musuh yang dapat terdampak oleh serangan area (AoE). • Memilih arah gerakan yang mendekatkan robot pada pusat serangan dengan nilai dampak terbesar. • Memberikan bonus besar pada area dengan cluster petak musuh atau netral yang padat. • Memberikan bonus sedang pada frontier wilayah kosong yang luas. • Memberikan penalti jika hasil AoE mengenai banyak petak yang sudah dimiliki sendiri tanpa membuka frontier baru. • Memberikan penalti tinggi jika setelah melakukan serangan sisa paint menjadi terlalu rendah.
Soldier	• Jika terdapat <i>ruin</i> yang terdeteksi, robot akan langsung bergerak menuju lokasi tersebut. • Jika sedang mengerjakan pembangunan <i>tower</i> atau <i>Special Resource Pattern (SRP)</i>, robot memilih gerakan yang meminimalkan waktu penyelesaian pola. • Jika tidak terdapat tugas ekonomi aktif, robot akan bergerak untuk menutup frontier wilayah di sekitar tower yang baru dibangun. • Petak musuh di sekitar <i>ruin</i>, tower, atau SRP memiliki prioritas target tertinggi. • Petak musuh pada jalur komunikasi cat sekutu menjadi prioritas kedua. • Petak musuh biasa diprioritaskan setelah kedua kondisi di atas.
Mopper	• Bergerak menuju area dengan banyak cat musuh di sekitar ruin, tower, atau SRP. • Memprioritaskan pembersihan wilayah yang penting bagi stabilitas ekonomi tim. • Bergerak ke jalur komunikasi cat sekutu yang sering rusak untuk memperbaiki konektivitas wilayah. • Mendukung robot sekutu yang memiliki paint rendah dengan melakukan transfer paint apabila diperlukan. • Membersihkan area inti tim dengan cepat ketika terdapat gangguan dari cat musuh.

3.3.5 Strategi Target Cat

Strategi target cat berfokus pada aksi pewarnaan yang memberikan peningkatan kontrol wilayah terbesar dengan biaya paint yang efisien.

Splasher

Splasher memilih pusat serangan yang memaksimalkan dampak area.

$$ScoreSplash(c) = 8E + 5N + 2F + 20R - 6A - 10W - 4Overpaint$$

Keterangan:

- E : jumlah petak musuh yang dapat ditimpa
- N : jumlah petak netral yang dapat diwarnai
- F : jumlah petak yang memperluas frontier wilayah
- R : bonus jika dekat ruin atau SRP strategis
- A : jumlah petak sekutu yang terbangun
- W : jumlah petak tidak berguna
- Overpaint : area yang sudah aman milik sendiri

Table 5: Prioritas strategi target cat untuk masing-masing robot

Robot	Prioritas Target Cat
Splasher	• Petak yang memungkinkan serangan area mengenai banyak petak musuh sekaligus. • Petak netral yang berada pada cluster besar sehingga memberikan ekspansi wilayah yang signifikan. • Petak pada frontier wilayah yang dapat membuka akses ekspansi ke area baru. • Petak musuh yang memutus konektivitas wilayah cat sekutu. • Petak netral biasa yang berada di sekitar posisi robot.
Soldier	• Petak yang merupakan bagian dari pola pembangunan tower pada lokasi <i>ruin</i>. • Petak yang merupakan bagian dari pola <i>Special Resource Pattern (SRP)</i>. • Petak musuh yang berada di sekitar ruin atau tower sekutu. • Petak musuh yang mengganggu jalur komunikasi cat sekutu. • Petak netral yang berada di sekitar posisi robot.
Mopper	• Petak yang dicat oleh musuh di sekitar tower sekutu. • Petak musuh yang berada di sekitar ruin atau SRP yang sedang dibangun. • Petak musuh yang memutus jalur konektivitas cat sekutu. • Petak musuh yang berada pada wilayah inti tim. • Petak musuh biasa yang berada di sekitar posisi robot.

3.3.6 Manajemen Sumber Daya

Early Game	Mid Game	Late Game
• stabilkan income chips dan paint • buka 1–2 garis ekspansi	• percepat coverage • amankan suplai paint untuk Splasher	• maximize map coverage • convert ekonomi jadi spam unit / refill paint / menjaga tiebreaker
• ruin dekat + aman → bangun tower • prefer Money Tower dulu jika chips menjadi bottleneck spawn • jangan terlalu banyak investasi SRP terlalu dini	• mulai tambah Paint Tower • bangun SRP jika area sekitar cukup aman dijaga 50 ronde • Mopper lebih aktif menjaga pola dan wilayah sekitar tower	• Money Tower tambahan hanya jika masih benar-benar menaikkan tempo spawn • kalau bottleneck utama sudah paint, ubah prioritas ke Paint Tower

Table 6: Strategi manajemen sumber daya berdasarkan fase permainan

Manajemen sumber daya dilakukan dengan menghitung kebutuhan tim pada setiap ronde.

- $NeedPaint = jumlah\ splasher\ aktif \times konsumsi\ paint$
- $NeedChips = target\ spawn\ rate \times biaya\ unit$
- $NeedDefense = jumlah\ ruin/SRP\ yang\ rawan$

Keputusan strategi:

- Jika chips rendah dan paint cukup → prioritaskan Money Tower
- Jika paint rendah dan unit aktif banyak → prioritaskan Paint Tower
- Jika area rawan besar → tambah Mopper
- Jika ekspansi lambat tetapi resource aman → tambah Splasher

3.3.7 Spawn Strategy

- Early game : Soldier lebih banyak untuk membuka ruin
- Mid game : Splasher menjadi unit dominan
- Jika banyak kontestasi ruin : jumlah Mopper meningkat
- Jika ekonomi paint stabil : produksi Splasher meningkat

3.3.8 Pemanfaatan Pola Khusus (SRP dan Tower)

Pemilihan ruin untuk pembangunan tower menggunakan skor berikut:

$$ScoreRuin = 50 - 2 \times Distance + 20 \times Safety + 25 \times ExpansionValue - 15 \times EnemyInterference$$

Prioritas pemilihan ruin:

1. jarak relatif dekat
2. area sekitar tidak didominasi cat musuh
3. tersedia paint untuk menyelesaikan pola
4. tidak terlalu banyak unit lain yang sedang mengerjakan
5. tower yang dihasilkan mendukung ekspansi selanjutnya

3.4 Alternatif Solusi Greedy 2

Strategi agresif berfokus pada ekspansi wilayah secara cepat dan penekanan terhadap wilayah musuh. Robot diprioritaskan untuk bergerak menuju area yang memiliki konsentrasi petak musuh atau petak netral yang dapat segera diambil alih.

Arah pergerakan robot ditentukan menggunakan fungsi heuristik berikut:

$$\begin{aligned} score(d) = & -10 \cdot dist^2(nextLoc(d), target) + \sum dist^2(nextLoc(d), ally_i) - \sum dist^2(nextLoc(d), enemy_j) \\ & + 8 \quad (\text{jika tile EMPTY}) + 20 \quad (\text{jika tile ENEMY_PRIMARY atau ENEMY_SECONDARY}) \end{aligned}$$

Heuristik ini memberikan prioritas pergerakan menuju area yang mengandung petak musuh atau petak netral, sehingga robot dapat secara agresif memperluas wilayah tim sekaligus menekan wilayah lawan.

Early Phase	
Unit	Strategi
Tower (Round $\leq$ 300)	Fokus memproduksi robot <i>Soldier</i> . Robot Soldier diarahkan menuju lokasi kandidat tower musuh untuk menghancurkan tower tersebut sejak awal permainan.
Soldier (Round $\leq$ 300)	Bergerak menuju dua tower awal milik musuh dengan tujuan menghancurkan tower tersebut pada fase awal permainan.
Splasher (300 $\leq$ round $\leq$ 600)	Berfokus mengambil wilayah pada area tengah peta serta mengurangi persentase wilayah yang dimiliki oleh musuh.
Mopper (300 $\leq$ round $\leq$ 600)	Mencari robot musuh di sekitar wilayah permainan dan melakukan serangan untuk mengganggu ekspansi wilayah lawan.

Table 7: Strategi agresif pada Early Phase

Mid Phase	
Unit	Strategi
Tower (Round $\leq$ 800)	Tower memproduksi semua jenis robot dengan komposisi rasio sekitar 1:2:1 untuk Soldier, Splasher, dan Mopper.
Soldier (Round $\leq$ 1000)	Fokus bergerak menuju robot musuh untuk melakukan penyerangan. Jika tidak terdapat musuh di sekitar wilayah tersebut, robot akan kembali menggunakan strategi seperti pada Early Phase.
Splasher (600 $\leq$ round $\leq$ 1000)	Berfokus untuk mengambil wilayah yang dimiliki musuh serta mencari area yang dapat diperluas.
Mopper (600 $\leq$ round $\leq$ 1000)	Mencari keberadaan robot musuh dan melakukan serangan untuk mengganggu kontrol wilayah lawan.

Table 8: Strategi agresif pada Mid Phase

Late Phase	
Unit	Strategi
Tower (Round $\geq$ 1000)	Tower memfokuskan produksi pada robot Splasher dan Mopper untuk membersihkan wilayah musuh serta mengubahnya menjadi wilayah milik tim.
Soldier (Round $\geq$ 1000)	Jika masih terdapat robot Soldier yang aktif, robot tersebut akan mencari lokasi <i>ruins</i> untuk membangun tower tambahan guna memperoleh sumber daya tambahan yang dapat digunakan untuk memproduksi robot Splasher dan Mopper.
Splasher	Bergerak menuju arah tower musuh untuk mencari tower yang masih bertahan. Jika tidak ditemukan, robot akan bergerak menuju sisi atau sudut wilayah musuh untuk memperluas kontrol wilayah tim.
Mopper	Bergerak menuju robot sekutu terdekat dan mengikuti pergerakannya untuk memberikan dukungan sekaligus membersihkan wilayah yang masih dikuasai musuh.

Table 9: Strategi agresif pada Late Phase

3.5 Alternatif Solusi Greedy 3

Strategi ini menggunakan pendekatan berbasis *state* yang ditentukan oleh beberapa faktor utama, yaitu jumlah paint yang dimiliki robot, jarak terhadap tower sekutu, posisi musuh, serta peluang untuk mewarnai wilayah secara efisien.

Robot akan menentukan tindakan yang diambil berdasarkan *state* yang sedang dialami, sehingga keputusan yang diambil bersifat adaptif terhadap kondisi permainan. Setiap *state* menentukan prioritas tindakan robot seperti melakukan ekspansi wilayah, mempertahankan wilayah, kembali ke tower untuk mengisi ulang paint, atau melakukan penarikan diri dari wilayah musuh.

3.5.1 State Robot

State	Deskripsi dan Perilaku
Refill	Aktif ketika $\text{paint} \leq 20\%$. Pada kondisi ini robot diperkirakan akan mengalami penalti besar jika tetap bergerak atau menyerang dalam beberapa ronde ke depan. • Bergerak menuju paint tower sekutu terdekat • Mendekati Mopper sekutu apabila berada di dekatnya • Bergerak menuju wilayah sekutu dan menjauhi wilayah musuh
Return to Tower	Aktif ketika $20\% \leq \text{paint} \leq 40\%$ dan robot berada jauh dari tower atau terlalu dalam di wilayah musuh. • Mundur menuju wilayah sekutu • Mengecat tile yang murah jika memungkinkan • Memprioritaskan tile yang dapat memperluas jalur sekutu menuju tower
Maintain	Aktif ketika jumlah paint cukup dan tidak ada kebutuhan mendesak untuk ekspansi atau pengisian ulang. • Mengecat tile kosong yang dapat membangun koridor aman • Memperluas area di sekitar tower • Membantu menyelesaikan pola pembangunan • Menghindari tile yang dapat membuat robot terisolasi
Expand	Aktif ketika $\text{paint} > 50\%$ dan terdapat jalur aman untuk bergerak maju. • Menargetkan tile netral yang mudah diambil • Bergerak ke frontier antara wilayah sekutu dan wilayah netral • Memperluas wilayah tanpa terlalu dalam memasuki area musuh • Menargetkan tile di sekitar ruin atau SRP
Contest Light	Aktif ketika terdapat cat musuh di dekat frontier dan robot memiliki peluang menyerang dengan paint yang cukup. • Menghapus atau menimpa tile musuh yang strategis • Memfokuskan pada pengurangan kontrol wilayah musuh • Menjaga efisiensi penggunaan paint
Retreat	Aktif ketika paint rendah dan terdapat musuh di sekitar atau ketika posisi robot berada di wilayah musuh. • Mundur menuju wilayah sekutu terdekat • Memilih tile dengan penalti penggunaan paint minimum • Menghindari kerumunan robot sekutu

Table 10: State robot dalam strategi greedy

3.5.2 Alur Setiap Robot

Robot	Peran dan Perilaku
Soldier	Berperan sebagai unit garis depan yang bertugas melakukan ekspansi wilayah secara aman. • Mengisi frontier wilayah sekutu • Membantu penyelesaian pola pembangunan • Menyerang tower musuh apabila biaya serangan relatif murah dan aman Perilaku berdasarkan paint: • Paint rendah: kembali ke tower • Paint sedang: mendukung frontier • Paint tinggi: melakukan ekspansi
Mopper	Berperan sebagai unit logistik utama yang menjaga stabilitas pasokan paint. • Mendistribusikan paint kepada robot sekutu • Membersihkan cat musuh pada frontier • Menjadi penghubung antara tower dan robot sekutu di garis depan Perilaku tambahan: • Transfer paint jika terdapat sekutu dengan paint rendah • Membersihkan tile musuh jika jumlahnya banyak • Mendekati Soldier atau Splasher yang jauh dari tower
Splasher	Unit situasional yang digunakan pada kondisi tertentu. • Tidak diproduksi pada awal permainan • Digunakan ketika terdapat cluster tile bernilai tinggi • Dipanggil ketika sumber daya tim sudah stabil • Bergerak bersama jalur suplai sekutu • Menyerang jika area cakupan serangan cukup besar

Table 11: Alur perilaku setiap jenis robot

3.5.3 Alur Robot Setiap Phase

Phase	Strategi
Early Game	• Menghindari ekspansi berlebihan • Membangun wilayah aman di sekitar tower • Membuka jalur paint yang stabil • Spawn unit secara hemat dan stabil (Soldier dan beberapa Mopper) • Memperluas radius wilayah aman • Menjaga robot tetap berada di atas batas aman paint
Mid Game	• Mengontrol frontier wilayah • Menjaga jalur suplai antara tower dan unit garis depan • Melakukan serangan ringan terhadap wilayah musuh • Membantu pembangunan pola tower atau SRP jika kondisi aman
Late Game	• Mempertahankan luas wilayah yang dimiliki • Menghindari kehilangan unit yang tidak perlu • Melakukan spawn unit secara kontinu • Menjaga kondisi tiebreaker seperti luas area, jumlah tower, total paint, dan jumlah unit hidup

Table 12: Alur strategi bot berdasarkan fase permainan

3.5.4 Threshold

Robot	Threshold Paint	Tindakan
Soldier	Paint > 50%	Boleh melakukan ekspansi wilayah
	$30\% < paint \leq 50\%$	Mendukung frontier yang aman
	$paint \leq 30\%$	Kembali menuju tower
	$paint \leq 15\%$	Wajib melakukan refill
Mopper	Paint > 60%	Mendukung frontier dan membersihkan wilayah musuh
	$25\% < paint \leq 60\%$	Tetap berada dekat robot sekutu
	$paint \leq 25\%$	Kembali ke tower untuk mengambil paint
Splasher	Paint > 65%	Boleh melakukan serangan AoE
	$paint \leq 65\%$	Menahan serangan agar tidak membuang paint secara tidak efisien

Table 13: Threshold keputusan robot berdasarkan jumlah paint

3.6 Analisis Efisiensi

Analisis berikut membahas efisiensi dan efektivitas dari setiap bot. Efisiensi berkaitan dengan penggunaan sumber daya komputasi maupun sumber daya permainan seperti paint dan chips, serta penggunaan bytecode pada setiap giliran. Sementara itu, efektivitas berkaitan dengan sejauh mana strategi tersebut mampu mencapai tujuan utama permainan, yaitu menguasai wilayah peta dan memenangkan pertandingan.

3.6.1 Alternatif Solusi Greedy 1

Bot pertama menggunakan strategi greedy yang berfokus pada pembangunan menara dan penguatan ekonomi tim. Berbeda dengan bot kedua yang memprioritaskan ekspansi agresif, bot ini lebih menekankan stabilitas jangka panjang melalui pembangunan tower pada lokasi ruins yang tersedia di peta.

Dari sisi efisiensi komputasi, strategi ini sedikit lebih kompleks karena robot harus mempertimbangkan lebih banyak kondisi sebelum memilih aksi. Robot perlu mendeteksi keberadaan ruins di sekitar peta, mengevaluasi apakah lokasi tersebut aman untuk pembangunan tower, serta menentukan apakah sumber daya yang dimiliki cukup untuk menyelesaikan pola pembangunan. Walaupun demikian, kompleksitas ini masih dapat dianggap efisien karena sebagian besar perhitungan tetap bersifat lokal. Robot hanya perlu memeriksa informasi mengenai ruins terdekat, kondisi wilayah di sekitar lokasi tersebut, serta jumlah paint yang dimiliki. Tidak ada pencarian global terhadap seluruh peta yang dapat menghabiskan bytecode secara signifikan. Dengan demikian, strategi ini masih berada dalam batas penggunaan bytecode yang wajar untuk setiap giliran.

Kelemahan dari sisi efisiensi muncul pada banyaknya mode atau state yang digunakan dalam strategi ini. Robot dapat berpindah antara beberapa mode seperti eksplorasi, pembangunan tower, pengisian ulang paint, atau pertahanan wilayah. Jika transisi antar mode ini tidak dikendalikan dengan baik, robot dapat menghabiskan beberapa ronde hanya untuk berpindah mode tanpa menghasilkan kontribusi signifikan terhadap kontrol wilayah. Misalnya, robot dapat berpindah dari mode eksplorasi ke mode pembangunan, kemudian kembali lagi ke eksplorasi sebelum pola pembangunan selesai. Meskipun demikian, secara keseluruhan strategi ini masih cukup efisien dalam penggunaan sumber daya permainan karena pembangunan tower secara konsisten akan meningkatkan produksi paint dan chips pada jangka panjang.

Dari sisi efektivitas, bot pertama ini dapat dianggap sebagai strategi yang paling stabil di antara ketiga alternatif yang dirancang. Pembangunan tower memberikan keuntungan jangka panjang yang signifikan karena tower tidak hanya berfungsi sebagai unit pertahanan tetapi juga sebagai sumber produksi sumber daya. Money tower akan meningkatkan jumlah chips yang tersedia untuk spawn robot baru, sementara paint tower akan meningkatkan suplai paint yang dapat digunakan oleh robot sekutu. Dengan bertambahnya jumlah tower yang dimiliki tim, kapasitas ekspansi wilayah juga meningkat karena robot dapat melakukan refill paint lebih sering tanpa harus kembali ke tower awal. Selain itu, tower juga

memberikan perlindungan tambahan terhadap serangan musuh sehingga wilayah yang sudah dikuasai menjadi lebih sulit direbut kembali oleh lawan. Walaupun strategi ini membutuhkan waktu lebih lama pada fase awal permainan untuk menyelesaikan pola pembangunan tower, keuntungan ekonomi yang dihasilkan akan sangat terasa pada fase mid game dan late game. Dengan ekonomi yang lebih kuat, tim dapat memproduksi lebih banyak robot serta mempertahankan wilayah dengan lebih stabil.

Namun, strategi ini juga memiliki beberapa kelemahan. Fokus yang terlalu besar pada pembangunan tower dapat menyebabkan ekspansi wilayah menjadi lebih lambat pada fase awal permainan. Jika lawan

menggunakan strategi agresif dan berhasil menguasai sebagian besar wilayah peta sejak awal, bot ini mungkin kesulitan untuk mengejar ketertinggalan tersebut.

3.6.2 Alternatif Solusi Greedy 2

Bot kedua menggunakan pendekatan greedy agresif yang memprioritaskan serangan terhadap musuh sebagai aksi utama pada setiap giliran. Strategi ini dirancang untuk menghasilkan tekanan tinggi pada wilayah lawan sejak awal permainan dengan cara mempercepat kontak antara robot sekutu dan wilayah musuh.

Dari sisi efisiensi komputasi, strategi ini relatif sederhana. Keputusan robot sebagian besar hanya didasarkan pada informasi lokal yang dapat diperoleh melalui fungsi sensing pada radius penglihatan robot. Robot tidak perlu melakukan evaluasi global terhadap seluruh peta, melainkan cukup memeriksa apakah terdapat musuh, wilayah netral, atau wilayah musuh di sekitar posisi saat ini. Pendekatan ini membuat proses evaluasi aksi menjadi cepat dan ringan sehingga penggunaan bytecode pada setiap giliran tetap rendah. Hal ini penting karena dalam Battlecode setiap robot memiliki batas penggunaan bytecode pada setiap ronde, dan jika batas tersebut terlampaui maka eksekusi program robot akan berhenti sebelum seluruh logika selesai dijalankan. Selain itu, pembagian strategi ke dalam tiga fase permainan yaitu early phase, mid phase, dan late phase juga memberikan keuntungan dari sisi efisiensi desain. Dengan membagi permainan berdasarkan jumlah ronde, setiap robot dapat menggunakan aturan yang lebih sederhana pada masing-masing fase tanpa harus melakukan perhitungan kompleks mengenai kondisi permainan secara keseluruhan. Pendekatan ini membuat implementasi program menjadi lebih modular dan lebih mudah dipahami.

Namun, dari sisi efisiensi penggunaan sumber daya permainan, bot ini memiliki beberapa kekurangan. Heuristik yang digunakan untuk memilih aksi cenderung terlalu kasar sehingga tidak selalu mempertimbangkan kondisi sumber daya secara optimal. Misalnya, pemilihan unit yang di-spawn oleh tower sebagian besar ditentukan oleh fase permainan, bukan oleh kebutuhan aktual di lapangan. Selain itu, pengelolaan cadangan paint pada robot tidak terlalu diperhatikan secara eksplisit. Akibatnya, robot sering tetap bergerak atau menyerang secara agresif meskipun cadangan paint sedang rendah. Padahal dalam aturan permainan, robot yang memiliki paint rendah akan mengalami peningkatan cooldown pada aksi tertentu. Hal ini dapat menyebabkan robot menjadi tidak efektif dalam melakukan ekspansi wilayah atau serangan. Dengan kata lain, walaupun strategi ini efisien dari sisi komputasi, efisiensi penggunaan sumber daya permainan masih dapat ditingkatkan.

Dari sisi efektivitas, bot kedua memiliki beberapa keunggulan pada fase awal permainan. Strategi agresif memungkinkan robot untuk dengan cepat menjangkau wilayah musuh dan mulai mengganggu kontrol wilayah lawan. Pada peta yang relatif terbuka dan tidak terlalu banyak rintangan, bot ini dapat menyebar dengan cepat dan menekan wilayah lawan sebelum mereka sempat membangun struktur atau menyiapkan pertahanan yang kuat. Selain itu, karena peta pada Battlecode bersifat simetris, strategi ini sering dapat memanfaatkan informasi mengenai posisi awal musuh untuk langsung mengarahkan robot menuju lokasi yang kemungkinan besar menjadi pusat aktivitas lawan. Hal ini meningkatkan peluang terjadinya konflik awal yang dapat menguntungkan tim jika berhasil memenangkan pertarungan tersebut.

Namun, efektivitas strategi ini cenderung rendah karena disebabkan oleh kurangnya pengelolaan ekonomi dan logistik dalam strategi tersebut. Pembangunan menara tidak selalu menjadi prioritas, sehingga kapasitas produksi paint dan chips tim dapat tertinggal dibandingkan lawan yang lebih fokus pada pembangunan struktur ekonomi. Selain itu, peran robot Mopper dalam strategi ini relatif terbatas dan lebih bersifat reaktif, yaitu

hanya membersihkan cat musuh di sekitar wilayah sekutu tanpa berperan aktif dalam menjaga suplai paint bagi robot garis depan. Akibatnya, meskipun bot ini dapat memberikan tekanan tinggi pada awal permainan, performanya sering kali menurun pada fase mid game dan late game ketika lawan sudah memiliki ekonomi yang lebih stabil serta produksi unit yang lebih konsisten. Dalam kondisi tersebut, bot ini dapat kesulitan mempertahankan wilayah yang telah diperoleh sebelumnya.

3.6.3 Alternatif Solusi Greedy 3

Bot ketiga menggunakan pendekatan greedy yang berfokus pada efisiensi penggunaan sumber daya dalam setiap keputusan. Strategi ini mengadopsi pendekatan berbasis state di mana robot menentukan tindakan berdasarkan kondisi internal seperti jumlah paint, posisi relatif terhadap tower sekutu, keberadaan musuh di sekitar, serta peluang ekspansi wilayah.

Dari sisi efisiensi komputasi, bot ini merupakan yang paling kompleks di antara ketiga strategi yang dikembangkan. Setiap robot perlu mengevaluasi berbagai kemungkinan aksi berdasarkan beberapa faktor sekaligus, seperti keuntungan wilayah yang diperoleh, biaya paint yang dikeluarkan, risiko terhadap serangan musuh, serta keberlanjutan posisi robot pada beberapa ronde berikutnya. Meskipun demikian, strategi ini tetap dirancang agar sesuai dengan batas bytecode yang ditetapkan oleh permainan. Sebagian besar evaluasi keputusan hanya menggunakan data lokal yang diperoleh dari fungsi sensing pada ronde tersebut. Tidak ada penggunaan algoritma pencarian yang kompleks seperti pathfinding global atau perhitungan optimasi yang memerlukan iterasi besar. Selain itu, beberapa optimasi juga diterapkan untuk mengurangi biaya komputasi, seperti menyimpan informasi mengenai lokasi ruin dalam cache untuk beberapa ronde serta membatasi target ekspansi robot hanya pada area tertentu. Dengan pendekatan ini, kompleksitas keputusan tetap dapat dikendalikan sehingga robot tidak melampaui batas bytecode yang tersedia.

Dari sisi efektivitas, strategi ini memiliki pendekatan yang sangat berhati-hati dalam menggunakan sumber daya. Robot akan selalu mempertimbangkan rasio antara keuntungan yang diperoleh dan biaya yang harus dikeluarkan sebelum melakukan suatu aksi. Hal ini membuat penggunaan paint menjadi lebih hemat dan mengurangi risiko robot kehabisan paint di wilayah musuh.

Namun, fokus yang terlalu besar pada efisiensi lokal ini juga menimbulkan beberapa kelemahan. Karena robot cenderung memilih aksi yang paling efisien secara langsung, mereka sering menghindari tindakan yang memiliki risiko lebih tinggi tetapi berpotensi memberikan keuntungan strategis yang lebih besar. Misalnya, robot mungkin memilih untuk memperluas wilayah secara perlahan di sekitar area aman daripada melakukan penetrasi lebih dalam ke wilayah musuh. Akibatnya, tekanan terhadap wilayah lawan menjadi relatif lebih rendah dibandingkan dengan strategi agresif pada bot pertama. Selain itu, pembangunan tower juga tidak selalu menjadi prioritas utama dalam strategi ini karena robot sering lebih fokus pada optimalisasi penggunaan paint pada wilayah yang sudah dikuasai.

Secara keseluruhan, strategi ini sangat baik dalam menjaga stabilitas sumber daya dan meminimalkan kerugian unit. Namun, dalam beberapa situasi permainan, strategi ini dapat menjadi terlalu defensif sehingga kehilangan momentum ekspansi wilayah yang penting untuk memenangkan permainan.

3.7 Strategi yang Dipilih

Dari tiga alternatif strategi greedy yang telah dirancang dan dianalisis sebelumnya, strategi yang dipilih untuk diimplementasikan dalam program adalah strategi pada Alternatif Solusi Greedy 1, yaitu strategi

greedy yang berfokus pada pembangunan menara (tower) dan penguatan ekonomi tim.

Pemilihan strategi ini didasarkan pada beberapa pertimbangan utama, seperti stabilitas permainan, efisiensi penggunaan sumber daya, serta efektivitas dalam jangka panjang. Dari sisi efektivitas permainan, strategi Alternatif 1 dinilai lebih mampu memberikan performa yang stabil sepanjang permainan. Berbeda dengan Alternatif 2 yang lebih agresif dalam melakukan ekspansi wilayah sejak awal permainan, Alternatif 1 menempatkan pembangunan tower sebagai prioritas utama ketika menemukan lokasi ruins yang tersedia. Tower memiliki peran penting dalam permainan karena dapat menghasilkan sumber daya tambahan seperti paint dan chips, serta berfungsi sebagai titik pertahanan terhadap serangan musuh. Dengan meningkatnya jumlah tower yang dimiliki tim, kapasitas produksi sumber daya akan meningkat sehingga tim dapat memproduksi lebih banyak robot serta mempertahankan wilayah dengan lebih baik.

Secara efisiensi penggunaan sumber daya, strategi Alternatif 1 memberikan pengelolaan sumber daya yang lebih terstruktur. Pembangunan tower memungkinkan tim memperoleh suplai paint dan chips secara berkelanjutan sehingga robot tidak terlalu sering mengalami kekurangan sumber daya saat melakukan ekspansi wilayah atau serangan. Selain itu, keberadaan tower juga mempermudah robot untuk melakukan pengisian ulang paint tanpa harus kembali ke titik awal, sehingga waktu dan langkah pergerakan robot dapat digunakan secara lebih efisien.

Strategi Alternatif 1 dinilai memiliki keseimbangan antara ekspansi wilayah dan pembangunan infrastruktur tim. Walaupun pembangunan tower membutuhkan waktu pada fase awal permainan, keuntungan ekonomi yang dihasilkan akan memberikan dampak yang signifikan pada fase permainan selanjutnya. Dengan ekonomi yang lebih kuat, tim dapat memproduksi robot secara lebih konsisten serta memperluas wilayah dengan lebih stabil.

Strategi ini juga lebih adaptif terhadap berbagai kondisi peta. Pada peta yang memiliki banyak ruins, Alternatif 1 dapat memanfaatkan peluang tersebut untuk membangun lebih banyak tower dan memperkuat kontrol wilayah secara bertahap. Hal ini memberikan keuntungan strategis dibandingkan pendekatan yang hanya berfokus pada ekspansi agresif tanpa mempertimbangkan pembangunan ekonomi.

Berdasarkan pertimbangan tersebut, strategi greedy pada Alternatif 1 dipilih sebagai strategi utama yang akan diimplementasikan dalam program. Strategi ini dinilai memberikan kombinasi yang lebih seimbang antara efisiensi penggunaan sumber daya, efektivitas penguasaan wilayah, serta stabilitas performa sepanjang permainan.

4 Implementasi dan Pengujian

4.1 Pseudocode

4.2 Struktur Data dan Method

4.2.1 G.java

Table 14: Struktur Data di G.java

Nama	Penjelasan
<code>RobotController rc</code>	Referensi utama ke API Battlecode untuk semua aksi dan sensor robot.
<code>int mapWidth, mapHeight, mapArea</code>	Dimensi peta yang disimpan saat inisialisasi agar tidak perlu di-query ulang tiap ronde.
<code>MapLocation mapCenter</code>	Titik tengah peta, digunakan sebagai tujuan fallback saat robot tidak tahu harus ke mana.
<code>Team team, opponentTeam</code>	Referensi tim sendiri dan tim musuh agar perbandingan tim tidak perlu memanggil <code>rc</code> setiap saat.
<code>int roundSpawned</code>	Ronde saat robot ini pertama kali di-spawn, untuk menghitung usia robot.
<code>Direction[] DIRECTIONS[8]</code>	Array 8 arah mata angin (tanpa CENTER) untuk iterasi pergerakan dan scanning.
<code>Direction[] ALL_DIRS[9]</code>	Array 9 arah termasuk CENTER, jarang dipakai dalam kode saat ini.
<code>MapLocation me</code>	Lokasi robot saat ini, di-update setiap ronde agar tidak memanggil <code>rc.getLocation()</code> berulang.
<code>int round</code>	Nomor ronde saat ini, di-update di awal setiap giliran.
<code>RobotInfo[] nearbyAllies</code>	Cache daftar robot sekutu dalam jangkauan sensor, di-refresh tiap ronde.
<code>RobotInfo[] nearbyEnemies</code>	Cache daftar robot musuh dalam jangkauan sensor, di-refresh tiap ronde.
<code>MapInfo[] nearbyMapInfos</code>	Cache informasi tile-tile di sekitar robot (paint, mark, wall), di-refresh tiap ronde.
<code>MapLocation[] nearbyRuins</code>	Cache daftar ruin yang terdeteksi di sekitar robot pada ronde ini.
<code>int[][] visited[30][30]</code>	Grid 30×30 yang mencatat ronde terakhir setiap sel 2×2 dikunjungi, untuk panduan eksplorasi.
<code>MapLocation lastKnownEnemyTower</code>	Lokasi terakhir tower musuh yang pernah terlihat, digunakan Soldier untuk mode ATTACK.
<code>int lastEnemyTowerRound</code>	Ronde saat <code>lastKnownEnemyTower</code> terakhir diperbarui, untuk menentukan apakah info masih relevan.

Table 14: Struktur Data di G.java

Nama	Penjelasan
<code>int lastPaint</code>	Field yang dideklarasikan tapi tidak pernah digunakan di mana pun dalam kode.
<code>StringBuilder indicator</code>	Buffer teks indikator debug yang ditampilkan di atas robot setiap ronde.

Table 15: Function/Procedure di G.java

Function	Penjelasan
<code>void markVisited(MapLocation loc)</code>	Mencatat lokasi yang diberikan ke grid visited dengan nomor ronde saat ini.
<code>int getVisited(MapLocation loc)</code>	Mengembalikan ronde terakhir lokasi tersebut dikunjungi, atau 0 jika belum pernah.
<code>int cheby(MapLocation a, MapLocation b)</code>	Menghitung jarak Chebyshev (max dari selisih x dan y) antara dua lokasi.

4.2.2 RobotPlayer.java

Table 16: Function/Procedure di RobotPlayer.java

Function	Penjelasan
<code>void run(RobotController rc)</code>	Melakukan inisialisasi satu kali lalu menjalankan loop utama yang memanggil logika robot sesuai tipenya setiap ronde.
<code>void updateRound()</code>	Memperbarui semua field global di G (ronde, lokasi, cache sensor, ruin) setiap awal giliran.

4.2.3 Soldier.java

Table 17: Struktur Data di Soldier.java

Nama	Penjelasan
<code>int TOWER_BUILD_CUTOFF = 600</code>	Ronde batas setelah itu Soldier berhenti membangun tower dan beralih ke ekspansi murni.
<code>int EXPLORE=0 ...ATTACK=6</code>	Konstanta integer yang merepresentasikan tujuh mode state machine Soldier.

Table 17: Struktur Data di Soldier.java

Nama	Penjelasan
<code>int mode</code>	State machine aktif saat ini, menentukan perilaku Soldier pada ronde ini.
<code>MapLocation targetRuin</code>	Ruin yang sedang menjadi target pembangunan tower oleh Soldier ini.
<code>UnitType targetType</code>	Tipe tower yang direncanakan akan dibangun pada <code>targetRuin</code> saat ini.
<code>boolean markerPlaced</code>	Flag yang menandakan apakah pola tower sudah ditandai (mark) pada ruin target.
<code>int buildStartRound</code>	Ronde saat proses pembangunan tower dimulai, untuk mengukur timeout.
<code>int BUILD_TIMEOUT = 110</code>	Batas maksimal ronde yang diperbolehkan untuk menyelesaikan satu proses pembangunan tower.
<code>UnitType[] TOWER_TYPES[3]</code>	Array tiga tipe tower level-1 yang digunakan saat iterasi pengecekan dan penyelesaian pola.
<code>Direction[] SENTINEL[3]</code>	Arah tile sentinel per tipe tower untuk memverifikasi bahwa pola sudah di-mark sebelum diselesaikan.
<code>boolean[][][] towerPatterns</code>	Pola boolean 5×5 untuk ketiga tipe tower, di-fetch saat init untuk digunakan dalam painting.
<code>boolean[][] resourcePattern</code>	Pola boolean resource pattern yang di-fetch saat init (tersimpan tapi tidak digunakan langsung).
<code>MapLocation exploreTarget</code>	Lokasi tujuan eksplorasi saat ini yang diperbarui saat target sudah tercapai atau kadaluarsa.
<code>int exploreAge</code>	Penghitung ronde sejak <code>exploreTarget</code> terakhir diperbarui, untuk timeout eksplorasi.
<code>int EXPLORE_TIMEOUT = 30</code>	Batas ronde sebelum <code>exploreTarget</code> dipilih ulang agar tidak stuck mengejar lokasi yang tidak bisa dicapai.

Table 18: Function/Procedure di Soldier.java

Function	Penjelasan
<code>void init()</code>	Mengambil pola tower dan resource dari rc, lalu menyetel seed PRNG robot ini.

Table 18: Function/Procedure di Soldier.java

Function	Penjelasan
<code>void run()</code>	Fungsi utama yang dijalankan tiap ronde: cek paint, jalankan transisi state, lalu eksekusi aksi mode aktif.
<code>void transitionExplore()</code>	Dari EXPLORE, mencari ruin terbaik untuk dibangun dan beralih ke GOTO_RUIN, atau ke ATTACK jika musuh terdeteksi.
<code>void transitionGotoRuin()</code>	Memvalidasi target ruin masih valid dan belum dicuri, lalu beralih ke MARK saat ruin sudah bisa disensor.
<code>void transitionMark()</code>	Menunggu <code>markerPlaced</code> menjadi true sebelum beralih ke mode PAINT.
<code>void transitionPaint()</code>	Memeriksa apakah pola sudah bisa diselesaikan dan beralih ke COMPLETE jika sudah siap.
<code>void transitionComplete()</code>	Mencoba menyelesaikan tower; jika gagal kembali ke mode PAINT.
<code>void transitionAttack()</code>	Kembali ke EXPLORE jika informasi tower musuh sudah terlalu lama atau tidak ada.
<code>void doExplore()</code>	Bergerak ke target eksplorasi dan mengecat tile di sekitar sepanjang jalan.
<code>void doGotoRuin()</code>	Bergerak menuju targetRuin sambil mengecat tile di sekitar sepanjang jalan.
<code>void doMark()</code>	Mendekati ruin dan memanggil <code>markTowerPattern</code> untuk menandai pola tower di peta.
<code>void doPaint()</code>	Bergerak ke tile pola yang paling mendesak dan mengecat tile sesuai pola tower.
<code>void doComplete()</code>	Mendekati ruin dan memanggil <code>tryCompleteTower</code> untuk menyelesaikan pembangunan tower.
<code>void doRetreat()</code>	Bergerak ke tower sekutu terdekat dan mengisi ulang paint dari tower tersebut.
<code>void doAttack()</code>	Bergerak ke lokasi tower musuh terakhir diketahui dan menyerangnya jika dalam jangkauan.
<code>boolean paintBestPatternTile()</code>	Memilih dan mengecat tile pola tower berprioritas tertinggi dalam jangkauan aksi.
<code>MapLocation findUrgentPatternTile()</code>	Menemukan tile pola yang belum dicat dan paling dekat dengan robot untuk dijadikan navigasi.

Table 18: Function/Procedure di Soldier.java

Function	Penjelasan
<code>boolean hasMopperNearby()</code>	Mengembalikan true jika ada Mopper sekutu dalam radius 8 dari posisi robot ini.
<code>void opportunisticComplete()</code>	Menyelesaikan pola tower mana pun yang sudah siap di sekitar robot tanpa harus dalam mode COMPLETE.
<code>boolean tryCompleteTower()</code>	Mencoba menyelesaikan tower pada targetRuin dengan memverifikasi sentinel tile terlebih dahulu.
<code>MapLocation findBestRuinToBuild()</code>	Memilih ruin terbaik untuk dibangun berdasarkan jarak, jumlah cat musuh, dan status klaim.
<code>int countEnemyPaint(MapLocation loc, int radiusSq)</code>	Menghitung jumlah tile bercat musuh dalam radius tertentu dari lokasi yang diberikan.
<code>MapLocation pickGreedyExploreTarget()</code>	Memilih target eksplorasi terbaik secara greedy dengan memprioritaskan ruin netral dan sel yang belum dikunjungi.
<code>void paintNearby()</code>	Mengecat tile terbaik dalam jangkauan aksi dengan memprioritaskan tile sendiri, tile kosong, lalu tile musuh.
<code>void tryCompleteResourcePatterns()</code>	Mencoba menyelesaikan resource pattern di semua tile dalam radius 2 dari posisi robot.
<code>void tryWithdrawPaint()</code>	Menarik paint dari tower paint terdekat jika robot berada dalam jangkauan dan paint belum penuh.
<code>MapLocation findNearestAllyTowerLoc()</code>	Menemukan tower sekutu terdekat dengan bonus preferensi pada paint tower untuk retreat.
<code>void checkRuinStolen()</code>	Memeriksa apakah ruin target sudah diambil musuh atau sekutu lain dan memperbarui state sesuai hasilnya.
<code>void abandonBuild()</code>	Mereset semua variabel build dan kembali ke mode EXPLORE.
<code>String modeStr()</code>	Mengembalikan string singkat mode aktif untuk ditampilkan di indikator debug.

4.2.4 Splasher.java

Table 19: Struktur Data di .java

Nama	Penjelasan
<code>int EXPLORE=0, RETREAT=1</code>	Dua konstanta mode state machine Splasher: eksplorasi aktif atau mundur ke tower.
<code>int mode</code>	Mode aktif Splasher saat ini.
<code>MapLocation exploreTarget</code>	Lokasi tujuan eksplorasi yang diperbarui secara berkala.
<code>int exploreAge</code>	Penghitung ronde sejak <code>exploreTarget</code> terakhir diganti.
<code>int EXPLORE.TIMEOUT = 25</code>	Batas ronde sebelum <code>exploreTarget</code> Splasher dipilih ulang.

Table 20: Function/Procedure di Splasher.java

Function	Penjelasan
<code>void init()</code>	Menyetel seed PRNG unik untuk robot Splasher ini.
<code>void run()</code>	Fungsi utama tiap ronde: cek paint, mode, complete pattern, lalu eksekusi aksi.
<code>void doExplore()</code>	Bergerak ke target terbaik dan melakukan splash attack pada lokasi paling menguntungkan.
<code>void bestSplash()</code>	Memilih pusat splash yang memberikan skor tertinggi berdasarkan jumlah tile kosong dan cat musuh dalam radius ledakan.
<code>MapLocation pickMoveTarget()</code>	Menilai 8 arah gerak dan memilih yang paling menguntungkan menuju area belum dikunjungi.
<code>MapLocation pickGreedyTarget()</code>	Memilih target eksplorasi acak terbaik dari 8 kandidat berdasarkan skor kunjungan dan jarak.
<code>void doRetreat()</code>	Bergerak ke paint tower terdekat dan menarik paint saat sudah berada dalam jangkauan.
<code>void tryWithdrawPaint()</code>	Menarik paint dari paint tower sekutu terdekat jika berada dalam jangkauan dan kapasitas belum penuh.
<code>void tryCompletePatterns()</code>	Menyelesaikan pola tower dan resource pattern yang sudah siap di sekitar robot secara oportunistik.
<code>MapLocation findNearestPaintTowerLoc()</code>	Mencari paint tower sekutu yang paling dekat dari posisi robot saat ini.

4.2.5 Mopper.java

Table 21: Struktur Data di Mopper.java

Nama	Penjelasan
<code>int EXPLORE=0, SUPPORT=1, RETREAT=2</code>	Tiga konstanta mode state machine Mopper.
<code>int mode</code>	Mode aktif Mopper saat ini.
<code>MapLocation exploreTarget</code>	Lokasi tujuan eksplorasi Mopper yang diperbarui secara berkala.
<code>int exploreAge</code>	Penghitung ronde sejak <code>exploreTarget</code> Mopper terakhir diganti.
<code>int EXPLORE_TIMEOUT = 20</code>	Batas ronde sebelum <code>exploreTarget</code> Mopper dipilih ulang.

Table 22: Function/Procedure di Mopper.java

Function	Penjelasan
<code>void init()</code>	Menyetel seed PRNG unik untuk robot Mopper ini.
<code>void run()</code>	Fungsi utama tiap ronde: cek paint untuk retreat, complete pattern, lalu eksekusi mode aktif.
<code>void doExplore()</code>	Mengecek ally yang butuh paint, bergerak ke area cat musuh terpadat, dan melakukan aksi mop.
<code>Direction pickMopDirection()</code>	Memilih arah gerak terbaik dengan menghitung skor berdasarkan kepadatan cat musuh dan robot musuh di sekitarnya.
<code>void doMopAction()</code>	Memilih aksi terbaik: swing jika banyak musuh dalam satu garis, atau mop tile/robot musuh terbaik.
<code>Direction findBestSwing()</code>	Menemukan arah kardinal terbaik untuk swing attack yang mengenai paling banyak robot musuh.
<code>RobotInfo findNeedyAlly()</code>	Mencari robot sekutu dalam radius 2 yang paint-nya di bawah 30% kapasitas untuk diberi support.
<code>void doSupport()</code>	Mendekati ally yang butuh paint dan mentransfer paint kepadanya.
<code>void doRetreat()</code>	Menarik paint dari paint tower terdekat atau bergerak menuju paint tower untuk isi ulang.
<code>void tryCompletePatterns()</code>	Menyelesaikan pola tower yang sudah siap di sekitar robot secara oportunistik.

Table 22: Function/Procedure di Mopper.java

Function	Penjelasan
<code>MapLocation findNearestPaintTowerLoc()</code>	Mencari paint tower sekutu yang paling dekat dari posisi Mopper saat ini.
<code>String modeStr()</code>	Mengembalikan string singkat mode aktif Mopper untuk indikator debug.

4.2.6 Tower.java

Table 23: Struktur Data di Tower.java

Nama	Penjelasan
<code>int PHASE2_ROUND = 601</code>	Ronde di mana strategi spawn beralih ke mode agresif fase 2 (dominasi Splasher).
<code>double W_SOLDIER.BASE = 1.6</code>	Bobot dasar Soldier dalam kalkulasi proporsi spawn fase 1.
<code>double W_SPLASHER.BASE = 0.5</code>	Bobot dasar Splasher dalam kalkulasi proporsi spawn fase 1.
<code>double W_MOPPER.BASE = 0.9</code>	Bobot dasar Mopper dalam kalkulasi proporsi spawn fase 1.
<code>double dSoldier, dSplasher, dMopper</code>	Akumulator desimal untuk menyeimbangkan jumlah spawn aktual vs target proporsi setiap tipe unit.
<code>int spawnedTotal, spawnedSoldiers, ...</code>	Counter jumlah unit yang sudah di-spawn oleh tower ini per tipe dan total.
<code>int lastSpawnRound</code>	Ronde terakhir tower berhasil men-spawn unit, untuk logika throttling spawn.
<code>MapLocation[] spawnLocs</code>	Array lokasi spawn yang sudah diurutkan dari yang terjauh dari pusat peta ke terdekat.
<code>int towerLevel</code>	Level upgrade tower saat ini, bertambah tiap kali <code>upgradeTower</code> berhasil dipanggil.

Table 24: Function/Procedure di Tower.java

Function	Penjelasan
<code>void init()</code>	Membangun daftar lokasi spawn dan mendaftarkan tower ini ke MapMemory.

Table 24: Function/Procedure di Tower.java

Function	Penjelasan
<code>void run()</code>	Fungsi utama tiap ronde: putuskan unit yang di-spawn, lakukan spawn, serang musuh, dan coba upgrade.
<code>UnitType decideSpawnPhase1()</code>	Memilih tipe unit terbaik untuk di-spawn di fase 1 berdasarkan bobot dinamis dan jumlah tower yang ada.
<code>UnitType decideSpawnPhase2()</code>	Memilih tipe unit untuk fase 2 dengan pola sederhana: setiap unit ke-7 adalah Mopper, sisanya Splasher.
<code>boolean shouldSpawn(UnitType t, boolean isPhase2)</code>	Menentukan apakah tower boleh spawn sekarang berdasarkan jumlah chip, jumlah tower, dan fase permainan.
<code>boolean trySpawn(UnitType type)</code>	Mencoba membangun robot di salah satu lokasi spawn yang tersedia dan memperbarui counter.
<code>void doAttack()</code>	Melakukan serangan AoE otomatis lalu menyerang musuh berprioritas tertinggi dalam jangkauan.
<code>void tryUpgrade()</code>	Mengupgrade tower jika chip mencukupi threshold dengan buffer sesuai fase permainan.
<code>MapLocation[] buildSpawnLocs()</code>	Membuat dan mengurutkan array 12 lokasi potensial spawn di sekitar tower berdasarkan jarak dari pusat peta.

4.2.7 Nav.java

Table 25: Struktur Data di Nav.java

Nama	Penjelasan
<code>Direction bugDir</code>	Arah dinding yang sedang diikuti dalam algoritma bug navigation, null jika sedang jalan lurus.
<code>int bugRot</code>	Arah rotasi bug nav: +1 searah jarum jam, -1 berlawanan jarum jam.
<code>int bugSteps</code>	Jumlah langkah yang sudah dilakukan dalam mode bug nav untuk deteksi timeout.

Table 25: Struktur Data di Nav.java

Nama	Penjelasan
MapLocation bugTarget	Target terakhir yang digunakan bug nav untuk mendeteksi perubahan tujuan.
int bugStartDist	Jarak kuadrat ke target saat bug nav dimulai, untuk mendeteksi kemajuan.
int BUG_RESET = 20	Batas langkah maksimal sebelum bug nav di-reset agar tidak berputar terus.

Table 26: Function/Procedure di Nav.java

Function	Penjelasan
void moveToward(MapLocation target)	Menggerakkan robot menuju target dengan bug navigation untuk menghindari rintangan.
void moveAway(MapLocation avoid)	Menggerakkan robot menjauhi lokasi yang diberikan dengan mencoba arah berlawanan dan variannya.
void spreadFromAllies()	Menggerakkan robot menjauh dari kerumunan ally dengan menghitung vektor repulsi.
boolean tryMove(Direction d)	Mencoba bergerak ke arah tertentu, memperbarui G.me dan visited jika berhasil, mengembalikan true/false.
void greedyExploreTo(MapLocation target)	Bergerak menuju target dengan memilih arah yang memaksimalkan kemajuan sekaligus mengutamakan sel belum dikunjungi.
boolean paintUnderSelf()	Mengecat tile di bawah robot sendiri jika tile tersebut kosong atau bercat musuh.
MapLocation pickExploreTarget()	Memilih target eksplorasi terbaik dari 8 kandidat acak berdasarkan skor kunjungan dan jarak.

4.2.8 MapMemory.java

Table 27: Struktur Data di MapMemory.java

Nama	Penjelasan
int MAX_RUINS = 100	Batas maksimal ruin/tower yang bisa disimpan dalam memori.
int numRuins	Jumlah ruin/tower yang saat ini tersimpan dalam array.

Table 27: Struktur Data di MapMemory.java

Nama	Penjelasan
<code>MapLocation[] ruinLocs[100]</code>	Array lokasi semua ruin dan tower yang pernah ditemukan.
<code>Team[] ruinTeams[100]</code>	Array tim pemilik setiap ruin/tower yang diketahui.
<code>UnitType[] ruinTypes[100]</code>	Array tipe tower untuk setiap ruin/tower yang sudah dibangun.
<code>int[] ruinStatus[100]</code>	Status setiap ruin: 0=netral, 1=milik sekutu, 2=milik musuh.
<code>int[] ruinClaimRound[100]</code>	Ronde saat robot mengklaim ruin untuk dibangun, untuk mencegah klaim ganda dalam 80 ronde.
<code>int allyPaintTowers</code>	Jumlah paint tower sekutu yang diketahui saat ini.
<code>int allyMoneyTowers</code>	Jumlah money tower sekutu yang diketahui saat ini.
<code>int allyDefenseTowers</code>	Jumlah defense tower sekutu yang diketahui saat ini.
<code>int totalAllyTowers</code>	Total semua tower sekutu yang diketahui saat ini.
<code>boolean[] symmetry[3]</code>	Tiga flag simetri peta yang masih mungkin berlaku: horizontal, vertikal, dan rotasional.
<code>int[][] towerGrid[12][12]</code>	Grid indeks 12×12 untuk lookup cepat apakah sudah ada ruin pada sel 5×5 tertentu.

Table 28: Function/Procedure di MapMemory.java

Function	Penjelasan
<code>void registerRuin(MapLocation loc, Team t, UnitType u)</code>	Menambahkan atau memperbarui data ruin/tower pada lokasi yang diberikan dan menyetel ulang hitungan tower.
<code>boolean tryClaimRuin(MapLocation loc)</code>	Mencoba mengklaim ruin netral untuk dibangun; gagal jika sudah diklaim dalam 80 ronde terakhir.
<code>int teamToStatus(Team t)</code>	Mengonversi objek Team menjadi integer status (0=netral, 1=sekutu, 2=musuh).
<code>void recomputeAllyCount()</code>	Menghitung ulang semua counter tower sekutu per tipe dari awal berdasarkan data <code>ruinTeams</code> dan <code>ruinTypes</code> .
<code>UnitType chooseTowerType()</code>	Memilih tipe tower terbaik untuk dibangun berikutnya berdasarkan rasio paint/money/defense yang ada.

Table 28: Function/Procedure di MapMemory.java

Function	Penjelasan
<code>void updateSymmetry(MapLocation loc, boolean isWall)</code>	Placeholder untuk memperbarui estimasi simetri peta berdasarkan tile yang dilihat, saat ini belum diimplementasi.
<code>MapLocation getEnemyMirror(MapLocation allyLoc)</code>	Memprediksi posisi tower musuh dengan mencerminkan posisi tower sekutu berdasarkan simetri yang paling mungkin.
<code>void scanNearby()</code>	Memindai semua ruina dan robot musuh di sekitar robot lalu mendaftarkannya ke memori peta.
<code>MapLocation closestNeutralRuina(MapLocation from)</code>	Mencari ruina netral terdekat dari titik yang diberikan berdasarkan jarak kuadrat.
<code>MapLocation closestEnemyTower(MapLocation from)</code>	Mencari tower musuh terdekat yang diketahui dari titik yang diberikan berdasarkan jarak kuadrat.

4.2.9 Rand.java

Table 29: Struktur Data di Rand.java

Nama	Penjelasan
<code>int state = 12345</code>	Seed dan state internal PRNG xorshift32 yang diubah setiap pemanggilan <code>next()</code> .

Table 30: Function/Procedure di Rand.java

Function	Penjelasan
<code>int next()</code>	Menghasilkan bilangan pseudorandom positif berikutnya menggunakan algoritma xorshift32.
<code>int nextInt(int bound)</code>	Mengembalikan bilangan pseudorandom dalam rentang $[0, bound)$ dengan memanggil <code>next()</code> lalu modulo.

4.3 Pengujian

4.4 Analisis Hasil



5 Kesimpulan dan Saran



Lampiran

Repository GitHub:

<https://github.com/...>

Video:

<https://...>



References

- [1] Thomas H. Cormen, *Introduction to Algorithms*, MIT Press.